

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	35% of CIA		SLOT B
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	B.Pranith Sai Nihal	Reg. No.:	192472294
Section / Batch:	AI	Date:	17-08-2026

Code of Conduct

I B.Pranith Sai Nihal / 192472294 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: B.Pranith

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. A company is developing an AI-powered customer support chatbot for banking services. The chatbot uses Context-Free Grammar (CFG) for parsing user queries. However, it faces issues such as:

Incorrect interpretation of ambiguous sentences

Failure to handle subject–verb agreement

Inefficient parsing for long queries

Example user query:

“Show me the transactions with the card from last month”

- a) Analyze the ambiguity present in the given sentence using CFG
- b) Evaluate the limitations of CFG in handling real-world conversational queries
- c) Evaluate and propose an improved solution using PCFG, feature structures, or Earley parsing
- d) Justify how your proposed method improves accuracy and efficiency in an industry setting

2. A voice assistant system (like Alexa/Siri-type applications) processes spoken commands using parsing techniques. The system currently uses top-down parsing, but faces:

Backtracking issues

Poor handling of incomplete or ambiguous input

Difficulty in real-time response

Example command:

“Book a flight to Delhi with a window seat”

- a) Analyze how the given sentence can lead to multiple parse structures
- b) Analyze the limitations of top-down parsing in this real-time application
- c) Analyze how Earley parsing can improve handling of ambiguity and partial inputs
- d) Compare and analyze the performance of top-down vs Earley parsing for such industry use cases

3. A healthcare company is developing an AI system to analyze patient medical reports and automatically extract critical information such as diagnosis, prescribed treatments, and follow-up actions.

The system must handle:

Complex and lengthy medical sentences

Ambiguity in sentence structure

Accurate subject-verb agreement

Relationships between medical actions and entities (sub-categorization)

Real-time processing for hospital use

Example input:

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

The current system, based only on basic Context-Free Grammar (CFG), fails to correctly interpret relationships and produces inconsistent outputs.

a) Design a complete NLP architecture integrating:

CFG for syntactic structure

Probabilistic CFG (PCFG) for ambiguity resolution

Feature Structures for agreement handling

Efficient parsing techniques

b) Develop a step-by-step workflow showing how the system processes the given medical sentence from input to structured output (diagnosis, action, location).

c) Create a method to:

Incorporate sub-categorization frames for medical verbs

Improve accuracy and scalability in a hospital environment

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1								
2								
3								

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ASSESSMENT TOOL 2 – INDUSTRY PROBLEM SOLVING TASK

Course Code: DSA03

Course Title: Natural Language Processing

Department: Artificial Intelligence and Data Science

CO Assessed: CO4 – Precision Assessment

Formulate context-free grammars, feature structures, probabilistic parsing techniques and construct parsing frameworks.

Q1. Banking Customer Support Chatbot Using CFG

a) Ambiguity in the Given Sentence Using CFG

The sentence is:

“Show me the transactions with the card from last month.”

A basic CFG can represent the sentence as:

$S \rightarrow VP$

$VP \rightarrow V NP$

$NP \rightarrow Det N PP$

$PP \rightarrow P NP$

One possible interpretation is:

Show

```

└── transactions
    ├── with the card
    └── from last month

```

This means:

Intent: Retrieve transactions

Instrument/Account: Card

Time: Last month

However, the sentence contains **prepositional phrase (PP) attachment ambiguity**.

The phrase **“with the card”** may modify:

1. **transactions** → transactions made using the card
2. **show me** → show the information using/associated with the card

The phrase **“from last month”** may also be attached to:

- transactions → transactions that occurred last month
- card → a card from last month, which is less likely but grammatically possible in a simple CFG.

A semantic interpretation can therefore be represented as:

QUERY(

Transaction,
Instrument = Card,
Time = LastMonth

)

The most likely banking interpretation is:

Retrieve transactions performed using the card during last month.

b) Limitations of CFG for Real-World Conversational Queries

A CFG is useful for representing syntactic structures, but a basic CFG has several limitations in banking applications:

1. **Ambiguity:** CFG may generate multiple valid parse trees without knowing which interpretation is intended.
2. **No probability:** All valid productions are normally treated equally.
3. **Weak agreement handling:** Basic CFG does not naturally encode features such as singular/plural agreement.
4. **Poor semantic understanding:** CFG primarily represents syntactic structure rather than customer intent.
5. **Long-query inefficiency:** Large grammars may generate many possible parses for lengthy queries.
6. **Limited conversational robustness:** Real users may use incomplete sentences, spelling variations, informal language, and disfluencies.
7. **Context is not represented:** A previous conversation such as "I lost my Visa card" can influence the meaning of "Show me the transactions," but a standalone CFG does not capture this context.

Therefore, CFG alone is insufficient for a production banking chatbot.

c) Improved Solution Using PCFG, Feature Structures and Earley Parsing

A better architecture combines **PCFG + Feature Structures + Earley Parsing**.

1. PCFG for Ambiguity Resolution

A Probabilistic CFG assigns probabilities to grammar productions:

[
P(A $\rightarrow$ \beta)
]

For example:

PP $\rightarrow$ with NP [0.70]

PP $\rightarrow$ from NP [0.90]

The parser can compare alternative parse trees and select the one with the highest probability.

For the example query, the system would favor:

transactions

├── with the card
└── from last month

because this interpretation is more probable in the banking domain.

2. Feature Structures for Agreement

Feature structures can represent grammatical information:

NP:

NUMBER = PLURAL

PERSON = 3

and:

VP:

NUMBER = PLURAL

PERSON = 3

The parser checks feature compatibility before accepting a parse.

For example:

The customer checks the balance.

NUMBER(customer) = singular

NUMBER(checks) = singular

while:

The customers checks the balance.

would fail the appropriate agreement constraint.

3. Earley Parsing for Efficient General CFG Parsing

Earley parsing is suitable for the chatbot because it can handle:

- Ambiguous grammars
- Left-recursive grammars
- Partial/incomplete input
- Long sentences
- General CFGs

It maintains states such as:

[$S \rightarrow \bullet VP, 0$]

[$VP \rightarrow V \bullet NP, 0$]

[$NP \rightarrow Det \bullet N, 0$]

and progressively advances the parse as tokens are processed.

d) Industry Justification

The proposed approach improves the banking chatbot in several ways:

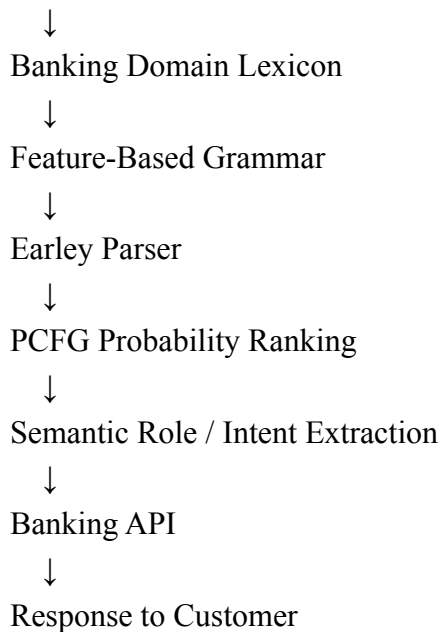
Problem	Proposed Technique	Improvement
Ambiguous parses	PCFG	Selects most probable interpretation
Subject–verb agreement	Feature structures	Rejects grammatically inconsistent parses
Long/complex queries	Earley parsing	Efficient chart-based parsing
Multiple interpretations	PCFG + semantic scoring	Selects contextually appropriate meaning
Banking terminology	Domain grammar	Improves domain-specific accuracy
Conversational input	NLP preprocessing + parser	Handles variations in user language

A practical production pipeline would be:

User Query

↓

Tokenization & Normalization



Conclusion: A hybrid PCFG, feature-structure and Earley-parsing approach is more suitable than a basic CFG because it combines syntactic validity, probabilistic ambiguity resolution, agreement checking and efficient parsing.

Q2. Voice Assistant Parsing

Command

“Book a flight to Delhi with a window seat.”

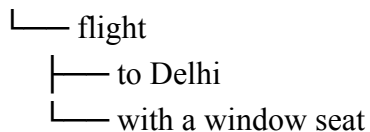
a) Multiple Possible Parse Structures

The sentence contains potential attachment ambiguity involving:

“with a window seat.”

A likely interpretation is:

Book



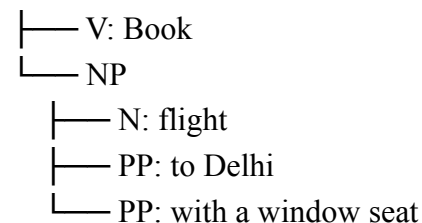
Semantic representation:

BOOK(
 Flight,
 Destination = Delhi,
 SeatPreference = Window
)

However, a parser could initially consider different attachment structures.

Parse 1 – Correct Interpretation

VP



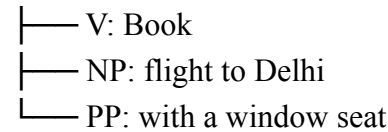
Meaning:

Book a flight whose destination is Delhi and whose seat preference is window.

Parse 2 – Alternative Attachment

The system could temporarily attach the final PP differently:

VP



Although the second structure may ultimately produce a similar interpretation, the parser still needs to determine the correct syntactic attachment.

Spoken language also introduces additional uncertainty because the system may receive:

- pauses,
- incomplete utterances,
- recognition errors,
- pronunciation variations,
- background noise.

b) Limitations of Top-Down Parsing

The current top-down parser begins from the start symbol and predicts productions before seeing all input.

This can cause:

1. **Backtracking:** Incorrect predictions may force the parser to return to earlier choices.
2. **Repeated work:** Similar substructures may be processed multiple times.
3. **Difficulty with ambiguity:** Several possible structures may cause extensive search.
4. **Poor handling of incomplete input:** Spoken commands may arrive incrementally.
5. **Real-time latency:** Excessive backtracking can delay the response.
6. **Speech-recognition uncertainty:** The parser may have to consider several possible word sequences.
7. **Scalability problems:** A large grammar containing many possible commands can increase search complexity.

For a voice assistant, latency is important because users expect a response immediately after issuing a command.

c) Earley Parsing for Ambiguous and Partial Input

Earley parsing uses a chart to store intermediate states and avoids repeatedly recomputing the same structures.

Its three major operations are:

1. Predictor

Predicts possible grammar rules.

[VP → • V NP, 0]

2. Scanner

Matches an input word.

[VP → V • NP, 0]

3. Completer

Completes a constituent and updates previously waiting states.

This allows the parser to maintain multiple hypotheses simultaneously.

For:

"Book a flight to Delhi with..."

the parser can retain partial structures while waiting for the remainder of the utterance.

When:

"a window seat"

arrives, the parser can incorporate it into the most appropriate structure.

This is particularly useful for real-time speech systems.

d) Top-Down vs Earley Parsing

Factor	Top-Down Parsing	Earley Parsing
Ambiguity	Can require extensive backtracking	Maintains alternative states
Partial input	Less suitable	Well suited
Left recursion	Problematic for simple implementations	Supported
Repeated computation	More likely	Reduced through chart storage
Real-time voice input	Can suffer from latency	Better suited to incremental processing
General CFG support	Depends on implementation	Strong
Memory usage	Potentially lower	Higher because of chart
Industry suitability	Suitable for small controlled grammars	Better for large, ambiguous grammars

For a general CFG, Earley parsing has worst-case cubic complexity:

[
 $O(n^3)$
]

and can perform considerably better for many practical grammars.

Industry Recommendation

The voice assistant should use:

Speech Input



Automatic Speech Recognition



Token / Hypothesis Generation



Earley Chart Parser



PCFG / Semantic Ranking



Intent + Slot Extraction



Flight Booking Service



Confirmation

The final semantic frame can be:

Intent = BOOK_FLIGHT

Destination = DELHI

SeatPreference = WINDOW

Conclusion: Earley parsing is better suited to voice assistants because it handles ambiguity and partial input without the extensive backtracking associated with simple top-down parsing.

Q3. Healthcare Semantic Processing Architecture

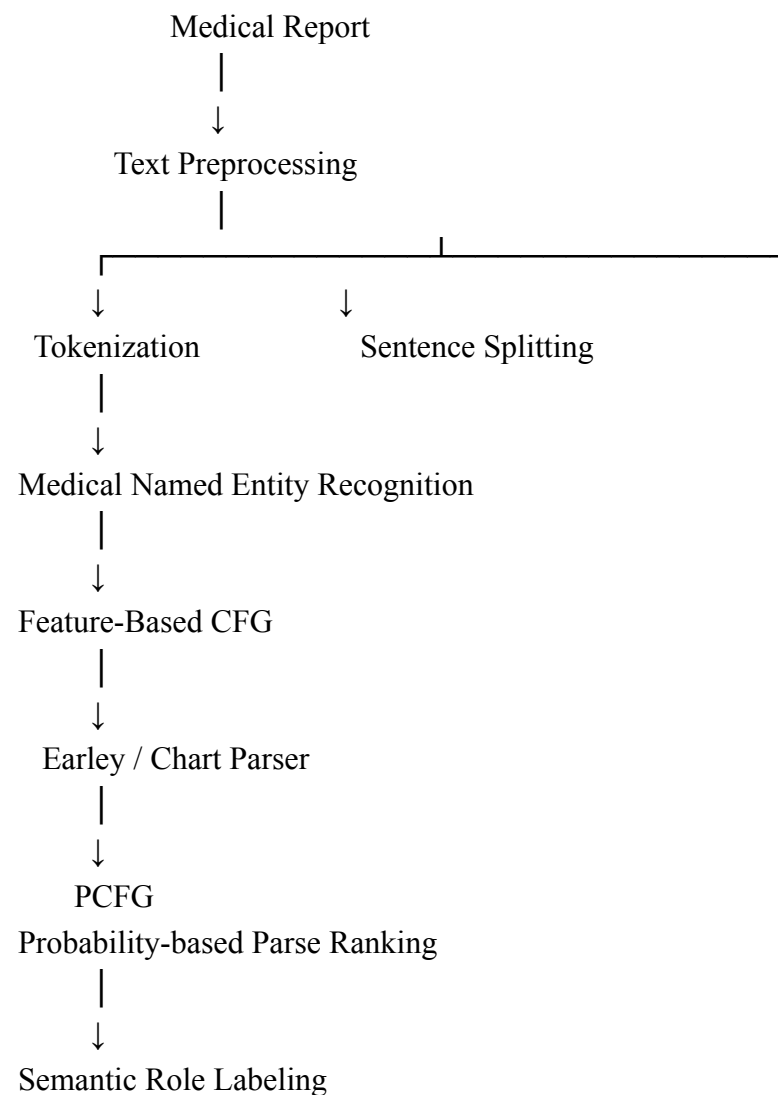
Given Medical Sentence

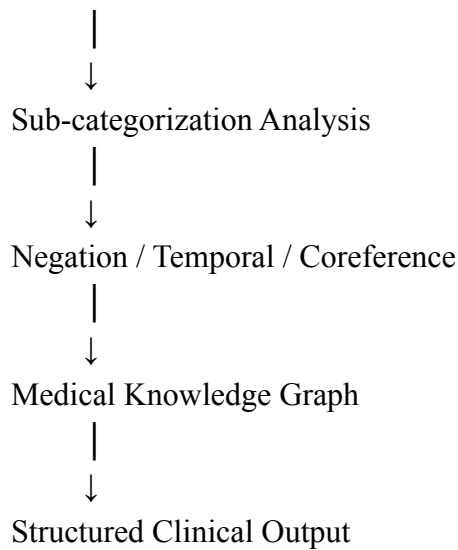
“The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.”

a) Complete NLP Architecture

A robust healthcare NLP system should combine CFG, PCFG, Feature Structures, efficient parsing and semantic processing.

Proposed Architecture





Role of Each Component

1. CFG

CFG establishes the basic syntactic structure:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$NP \rightarrow NP RelClause$

$VP \rightarrow V GerundPhrase Conj GerundPhrase PP$

It provides the initial syntactic framework.

2. PCFG

PCFG assigns probabilities to competing parses.

For example, the system can compare different attachments of:

“last week”

and:

“in Chennai.”

The most probable medically appropriate parse is selected.

3. Feature Structures

Feature structures encode grammatical properties:

Doctor:

PERSON = 3

NUMBER = SINGULAR

recommends:

PERSON = 3

NUMBER = SINGULAR

The agreement constraints match:

Doctor + recommends

Singular + Singular

Similarly:

doctor $\rightarrow$ who reviewed...

ensures that the relative clause is attached to the doctor.

4. Efficient Parsing

An Earley/chart parser can maintain partial structures and avoid repeatedly parsing the same constituents.

This is useful for lengthy clinical reports.

5. Semantic Processing

The semantic layer identifies:

- Medical entities
- Actions
- Agents
- Patients
- Treatments
- Temporal expressions
- Locations
- Follow-up activities

b) Step-by-Step Workflow

Step 1 – Input

The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.

Step 2 – Tokenization

The sentence is divided into tokens:

The | doctor | who | reviewed | the | patient | last | week |
recommends | starting | medication | and | scheduling | a |
follow-up | visit | in | Chennai

Step 3 – Entity Recognition

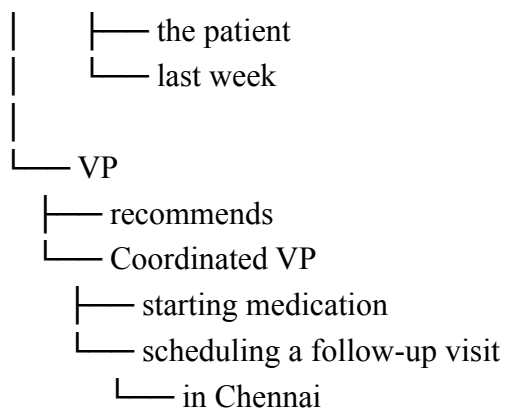
The system identifies:

Text	Entity Type
doctor	Medical professional
patient	Patient
medication	Treatment
follow-up visit	Medical procedure/ action
Chennai	Location

Step 4 – Syntactic Parsing

A simplified parse is:

S
├── NP
│ ├── The
│ ├── doctor
│ └── Relative Clause
│ ├── who
│ └── reviewed



Step 5 – Feature Agreement

The system verifies:

doctor = singular

recommends = singular

Therefore:

NUMBER(doctor) = NUMBER(recommends)

The agreement is valid.

Step 6 – PCFG Ranking

Potential syntactic interpretations are assigned probabilities.

The preferred interpretation is:

[doctor [who reviewed patient last week]]

[recommends

[starting medication]

and

[scheduling follow-up visit in Chennai]]

Step 7 – Semantic Role Assignment

Entity / Phrase	Role
Doctor	Agent / Recommender
Patient	Patient/Entity reviewed
Medication	Treatment
Starting medication	Recommended action
Scheduling follow-up visit	Recommended action
Chennai	Location
Last week	Temporal modifier

Step 8 – Temporal and Location Extraction

Temporal information:

last week

Location:

Chennai

The system should distinguish **"last week"** as the time of the review, not necessarily the time of the recommendation.

Step 9 – Structured Output

A possible structured representation is:

Doctor:

Entity = Doctor

Action = Reviewed

Target = Patient

Time = Last Week

Recommendation:

Agent = Doctor

Actions = [

Start Medication,

Schedule Follow-up Visit

]

Location = Chennai

Important Observation About Diagnosis

The sentence **does not explicitly contain a diagnosis**.

Therefore, the system must **not invent or infer a diagnosis** merely from the recommendation for medication.

The structured result should record:

Diagnosis = Not specified

This is important for clinical safety and semantic accuracy.

c) Sub-Categorization, Real-Time Processing and Scalability

1. Sub-Categorization Frames for Medical Verbs

Sub-categorization describes the types of arguments a verb expects.

Examples:

"Review"

review:

Agent = Doctor

Theme/Patient = Patient

Frame:

Doctor reviews Patient

"Recommend"

recommend:

Agent = Doctor

Action/Proposition = Treatment or Procedure

Frame:

Doctor recommends [starting medication]

"Start"

start:

Agent = Patient/Clinician

Theme = Medication

Frame:

[Patient] starts [Medication]

"Schedule"

schedule:

Agent = Doctor/Staff

Event = Follow-up Visit

Location = Chennai

Frame:

Staff schedules [follow-up visit] [in Chennai]

These frames allow the system to distinguish entities from actions and determine which arguments belong to each medical verb.

2. Real-Time Processing Method

For hospital use, the system should use a low-latency pipeline:

Incoming Report



Streaming Tokenization



Medical NER



Incremental Chart Parsing



PCFG Ranking



Semantic Extraction



Clinical Database / EHR

Recommended techniques include:

- Incremental Earley/chart parsing.
- Cached grammar rules.
- Medical-domain vocabulary.
- Pre-trained clinical language models.
- Parallel processing for independent documents.
- GPU acceleration where appropriate.
- Confidence scores for extracted information.
- Human review for low-confidence clinical information.

For example:

High confidence → Automatically structured

Medium confidence → Flag for verification

Low confidence → Human clinical review

This reduces the risk of incorrect information entering a patient's medical record.

3. Accuracy Improvement

Accuracy can be improved through:

Feature-Based Grammar

Use:

NUMBER

PERSON

TENSE

CASE

to enforce grammatical consistency.

PCFG

Use probability scores to select the most likely parse.

Medical Lexicon

Include specialized terms for:

- Diseases
- Drugs
- Procedures
- Symptoms
- Medical professionals
- Anatomical entities

Semantic Role Labeling

Identify:

Agent

Patient

Theme

Treatment

Time

Location

Negation Detection

The system must distinguish:

Patient has diabetes.

from:

Patient does not have diabetes.

Coreference Resolution

Resolve expressions such as:

The doctor examined the patient. He recommended medication.

so that the system determines who "**He**" refers to.

Temporal Reasoning

Distinguish:

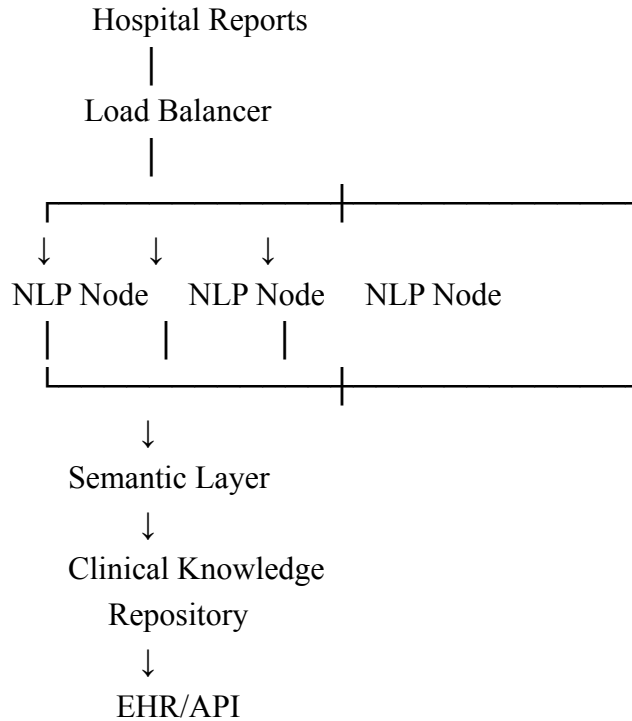
reviewed → last week

follow-up visit → future/recommended action

rather than attaching both expressions to the same event.

4. Scalability in a Hospital Environment

A scalable architecture can use:

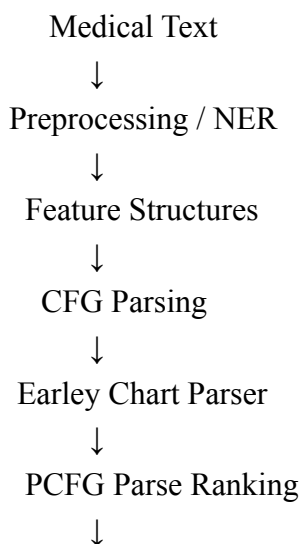


Important scalability measures include:

1. **Horizontal scaling** – add NLP processing nodes as report volume increases.
2. **Caching** – reuse frequently required grammar and terminology resources.
3. **Batch processing** – process large historical datasets efficiently.
4. **Streaming processing** – handle new hospital reports with low latency.
5. **Model optimization** – use smaller domain-specific models when appropriate.
6. **Monitoring** – track latency, parsing accuracy, entity extraction precision and failure rates.
7. **Security** – protect patient information through authentication, authorization and encryption.
8. **Human-in-the-loop validation** – require clinical verification for high-risk outputs.

Overall Proposed Healthcare Solution

The complete system can be summarized as:



Sub-Categorization Analysis



Semantic Role + Event Extraction



Negation + Coreference + Temporal NLP



Structured Clinical Data



Confidence / Validation



Hospital EHR

Final Structured Interpretation

For the given sentence:

Diagnosis = Not specified

Reviewer = Doctor

Reviewed Entity = Patient

Review Time = Last Week

Recommendation:

1. Start Medication
2. Schedule Follow-up Visit

Location = Chennai

The system should **not infer an unstated diagnosis**. The sentence only provides information about a doctor's review and recommendations.

Final Conclusion

A basic CFG provides a useful syntactic foundation but cannot adequately address ambiguity, agreement, probability, contextual meaning and the complexity of real-world medical language. An industry-grade NLP system should therefore integrate **CFG, PCFG, feature structures, Earley/chart parsing, sub-categorization frames and semantic role analysis**.

For banking, the combination improves interpretation of ambiguous customer requests. For voice assistants, Earley parsing reduces the limitations of backtracking and supports partial input. For healthcare, the integrated architecture provides accurate extraction of medical actions, entities, temporal information and locations while avoiding unsupported conclusions such as inventing a diagnosis.

Thus, a **hybrid symbolic-probabilistic NLP architecture** provides a practical balance between grammatical precision, ambiguity resolution, computational efficiency and real-world industry scalability.